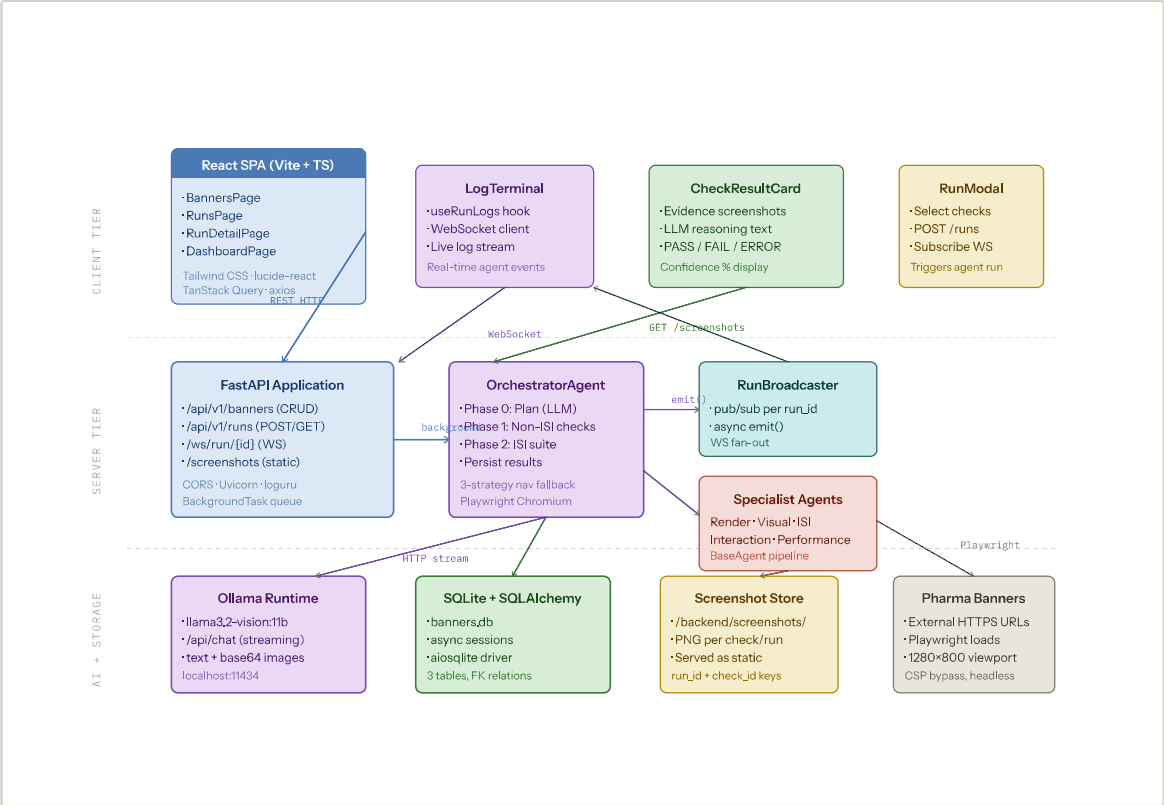
01

ARCHITECTURE TYPE — STRUCTURAL OVERVIEW

# High-Level System Overview

End-to-end picture of how BannerMind fits together — from the browser running the QA analyst's dashboard all the way down to the Ollama vision model and local filesystem. The three horizontal tiers (client, server, AI runtime) reflect actual deployment boundaries.




- REST / HTTP boundary
- Agent / orchestration layer
- Data / persistence
- Real-time (WebSocket)
- Specialist agents (check execution)
- File system / external target

DEPLOYMENT MODEL

Single-host deployment. All tiers run on the same machine. No service mesh or container orchestration required for development.

REAL-TIME COMMUNICATION

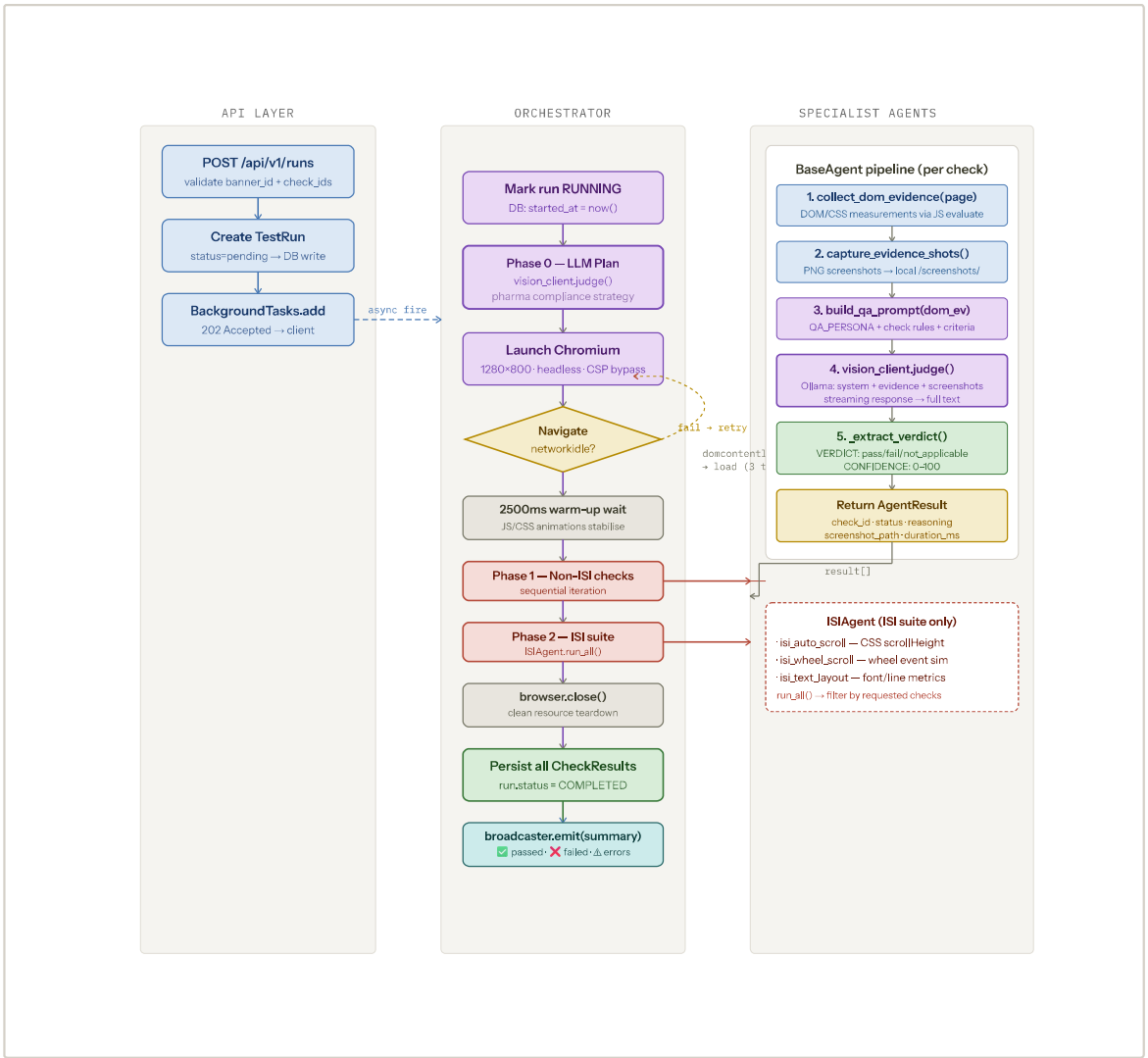
WebSocket at `/ws/run/{id}` fans out live agent log events to all connected clients for that run. No polling.

02

ARCHITECTURE TYPE — PROCESS FLOWCHART

# Multi-Agent Orchestration Pipeline

Execution trace of a single test run from the moment the POST /runs request arrives to the final database write. Covers the three-phase design (plan → non-ISI → ISI), the navigation fallback strategy, and how each specialist agent produces an AgentResult.




THREE-PHASE DESIGN RATIONALE

Non-ISI checks run first while the banner is fully rendered. ISI checks follow

NAVIGATION FALLBACK

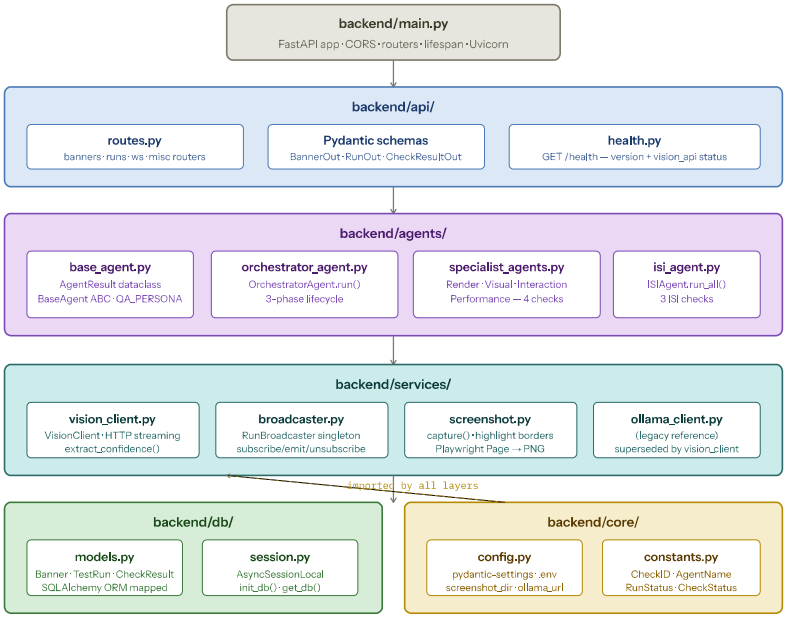
Pharma banners often have heavy JS payloads that cause `networkidle` to

03

ARCHITECTURE TYPE — MODULE DEPENDENCY MAP

# Backend Package Architecture

Python package structure and dependency direction across the five backend layers. Arrows indicate import relationships. The strict downward dependency rule keeps the agent layer testable without touching the API or DB layers.




DEPENDENCY DIRECTION

All imports flow downward. The API layer knows about agents; agents know about services; services know only about core config. Nothing in `core/` imports from any other package layer.

SERVICES AS ADAPTERS

`vision_client.py` and `broadcaster.py` are both module-level singletons, instantiated once and imported by agents. Swapping the vision backend means only changing this file.

CONSTANTS AS CONTRACT

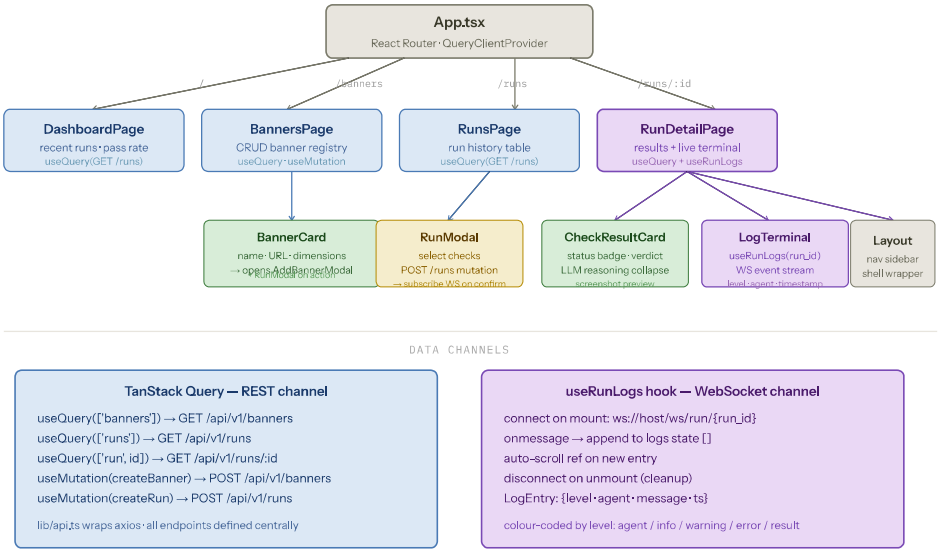
`CheckID`, `AgentName`, and status enums in `constants.py` form the shared

04

ARCHITECTURE TYPE — COMPONENT AND DATA-FLOW MAP

# Frontend Architecture & Data Flow

React component tree with the two data channels overlaid: TanStack Query handles all REST polling and mutations; the `useRunLogs` hook owns the WebSocket subscription lifecycle for live log streaming.




STATE MANAGEMENT

No global state store (no Redux/Zustand). TanStack Query's cache is the source of truth for server data. Component-local state handles UI concerns like modals and form inputs.

WEBSOCKET LIFECYCLE

The `useRunLogs` hook opens one WebSocket per run detail view and closes it on unmount. Multiple browser tabs watching the same run each get their own WS connection — the broadcaster fans out to all.

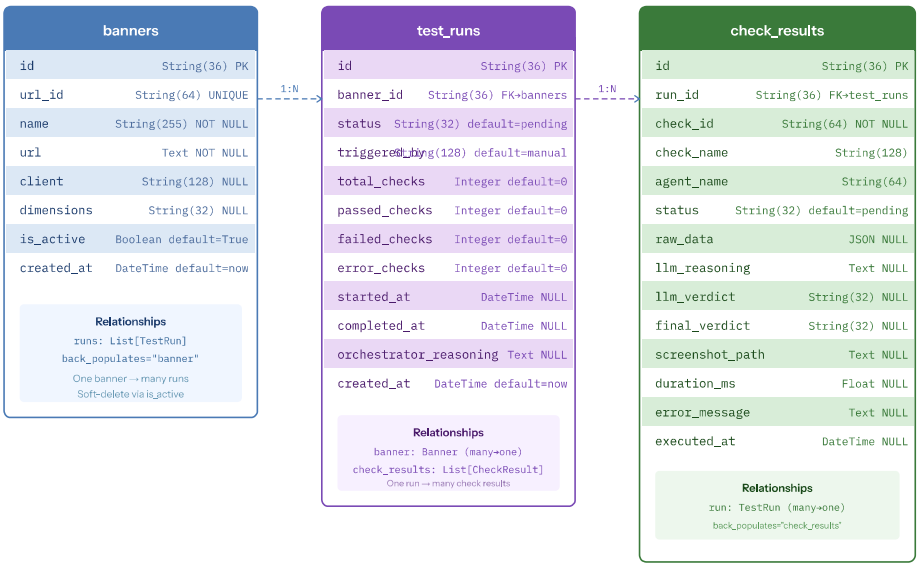
TYPE SAFETY

05

ARCHITECTURE TYPE — ENTITY RELATIONSHIP DIAGRAM

# Database Schema & Persistence Layer

The three-table SQLite schema that backs all banner registry, test run lifecycle, and per-check result data. Column types, nullability, and foreign key relationships shown. All async via SQLAlchemy 2.x + aiosqlite.




SCHEMA DESIGN NOTES

UUIDs as primary keys throughout. `url_id` is a human-readable slug for deduplication checks at the API layer, separate from the UUID primary key.

AUDIT COLUMNS

`llm_verdict` and `final_verdict` are stored separately. In the current version they are always equal, but the schema supports future signal arbitration logic where they could diverge.

RAW_DATA AS JSON

Each specialist agent stores the raw DOM evidence dictionary in the `raw_data` JSON column. This preserves

ASYNC RUNTIME

All DB sessions use `AsyncSessionLocal` backed by `aiosqlite`. No blocking I/O on the event loop. `init_db()` runs on